

SYSTEM PROPOSAL

ReqCheck: A Web-Based Personal Document and Requirement Readiness Management System

Platform: Web Application

Prepared by: Alvin Beato

Section: LFCA411E069

Submitted to
Noel Montecillo

Date: September 8, 2026

Table of Contents

1. Introduction
2. Purpose of the Project
3. Objectives
4. Project Scope
5. System Features / Functions
6. Technology Stack
7. Target Users
8. Project Timeline / Work Plan

1. Introduction

A person told to “bring these documents” for a service transaction is often left to work out, on their own, which documents are needed, whether an existing ID is still valid, whether a photocopy or a 1x1/2x2 photo is required, and whether one document must first be obtained before another can be completed. Requirements for a single service are frequently spread across different offices and instructions, and the requirement for one document sometimes depends on first securing another. The common result is a repeat visit: an applicant arrives at an office and only then discovers that one document is missing. This pattern is reflected in reports on service-delivery complaints for agencies such as the Social Security System (SSS), which cite branch queues and repeat visits for benefit claims as a recurring issue.

To address this, **ReqCheck** is proposed as a **web-based personal transaction-readiness system** designed to help citizens determine the documents, prerequisites, and preparation steps required for completing common services. Rather than functioning solely as a document checklist, the system analyzes the dependencies between requirements and generates a personalized preparation path based on the documents a user already has and the requirements of the service they select. Requirement information can also be consumed from more than one source — for example, government, barangay, and organization requirement sources normalized into a single requirement engine — so the system reflects genuine interoperability rather than having it added on afterward.

2. Purpose of the Project

The purpose of this project is to reduce unnecessary trips, incomplete applications, forgotten documents, expired documents, repeated transactions, and confusion regarding requirements. In effect, the system is meant to answer a single, relatable question for the user: **“If I go there tomorrow, am I actually ready?”**

3. Objectives

3.1 General Objective

To develop an interoperable system that determines whether a citizen is prepared to complete a selected service, based on document requirements and prerequisite dependencies.

3.2 Specific Objectives

- To create a centralized requirements model.
- To import requirements from different sources.
- To normalize different requirement formats.
- To detect missing or expired requirements.
- To identify dependencies between requirements.
- To provide service-specific checklists.
- To notify users about upcoming document expiration.
- To provide APIs for external systems.
- To maintain logs for requirement changes.

4. Project Scope

4.1 In Scope

- **Requirements Database** - Records, for every service, the service itself, its requirements, the documents involved, document validity, dependencies between requirements, and the issuing authority.
- **Document Inventory** - Allow users to record the documents they currently hold, such as a Government ID, Birth Certificate, Passport, Clearance, and similar records.
- **Requirements Checking** - Retrieves a selected service's requirements, compares them against the user's document inventory, checks document expiration, resolves dependencies between requirements, and generates a readiness result.

4.2 Out of Scope

- Upload or storage of the actual sensitive document itself - the system records only that user holds a given document (e.g., "I have this document"), rather than storing an actual government ID scan or similar file.

5. System Features / Functions

Module	Feature	Description
Readiness Evaluation	"Am I Ready?"	The core feature. When a user selects a service, the system returns a readiness percentage and a document-by-document breakdown — e.g., 82% Ready , with Valid ID, Birth Certificate, and Supporting Document satisfied, Clearance still missing and flagged as "must be obtained first," and a warning that the ID expires in 21 days.
ID Validation	ID Expiration Checking	Allow user to upload an image of a supported government-issued ID for automated expiration checking.
ID Validation	Expiration Date Extraction	Uses OCR to automatically extract the expiration date displayed on the uploaded ID.
ID Validation	Expiration Status Checking	Compares the extracted expiration date with the current date and determines whether the ID is valid or expired.
Dependency Management	Requirement Dependency Map	Shows why a requirement is still missing by displaying the chain of prerequisites behind it (e.g., Service → Clearance → Valid ID → Proof of Address), instead of presenting an unordered checklist.
Requirement Monitoring	Requirement Change Detection	Detects when an external requirement changes — for example, a photocopy requirement changing from 2 copies to 3 — and flags it to the user as a changed requirement, supporting data synchronization with outside requirement sources.
Requirement Transparency	Requirement Source	Displays the source of each requirement and when it was last verified (e.g., "Source: Agency X, Last verified: September 6, 2026"), so the system is transparent about how current its information is.

Module	Feature	Description
Notification	Expiration Tracking	Flags the validity status of each recorded document, such as a Passport marked Valid, a Government ID expiring in 27 days, or a Clearance marked Expired.
Preparation Guidance	Preparation Route	Instead of simply stating that documents are missing, generates an ordered sequence of steps — e.g., Step 1: obtain Document A → Step 2: use Document A to obtain Document B → Step 3: complete the application → Ready.

6. Technology Stack

Layer	Technology / Tool	Purpose
Frontend (Web)	Svelte + Tailwind CSS	User interface for the web application
Backend / API	Express.js (Node.js)	Server-side processing and the requirement/dependency engine
Database	PostgreSQL (via Supabase)	Managed PostgreSQL storage for users, projects, groups, responsibilities, tasks, evidence, and verification.
Communication	REST API	Communication between the frontend, backend, and external requirement sources
External Integration	Government / organization requirement sources	Retrieval of requirement data where it is legitimately available
OCR/Image Text Recognition	Tesseract.js	Extract text from uploaded ID images, particularly the expiration date, for automated validity checking.
Real-time	Optional	Live updates to requirement or readiness status
Authentication	JWT / Session-based authentication	Secure user login and access control
Other Tools	Figma	UI/UX mockups and wireframes prior to development

7. Target Users

User Role	Description	Key Needs / Usage
Primary Users	Ordinary citizens, particularly people preparing for government transactions, employment applications, permits, registrations, clearances, memberships, and applications requiring multiple documents.	Record documents they currently hold, check readiness for a selected service, and follow the generated preparation sequence.

8. Project Timeline / Work Plan

Phase	Activities	Estimated Duration
Planning & Requirements Gathering	Finalize service and requirement scope, define the requirement/dependency model, and prepare documentation	1 week
System Design	Database schema design, API design, dependency-engine design, and UI/UX wireframing	1 week
UI/UX Design & Mockups	Wireframing and high-fidelity mockups in Figma for the document inventory, readiness dashboard, dependency map, and preparation route screens	1 week
Backend Development	Build the Express.js REST API, requirement database, dependency resolver, expiration checks, and change-detection logic	2 weeks
Frontend Development	Build the Svelte + Tailwind CSS interface based on the approved Figma mockups: document inventory, "Am I Ready?" dashboard, dependency map, and preparation route views	2 weeks
Integration & Testing	Connect frontend to backend, end-to-end testing of the readiness engine, bug fixing, and refinement	1.5 weeks
Deployment	Hosting setup, database deployment, and final system configuration	3 days
Documentation & Final Presentation	Finalize project documentation and prepare for system demonstration and defense	4 days